

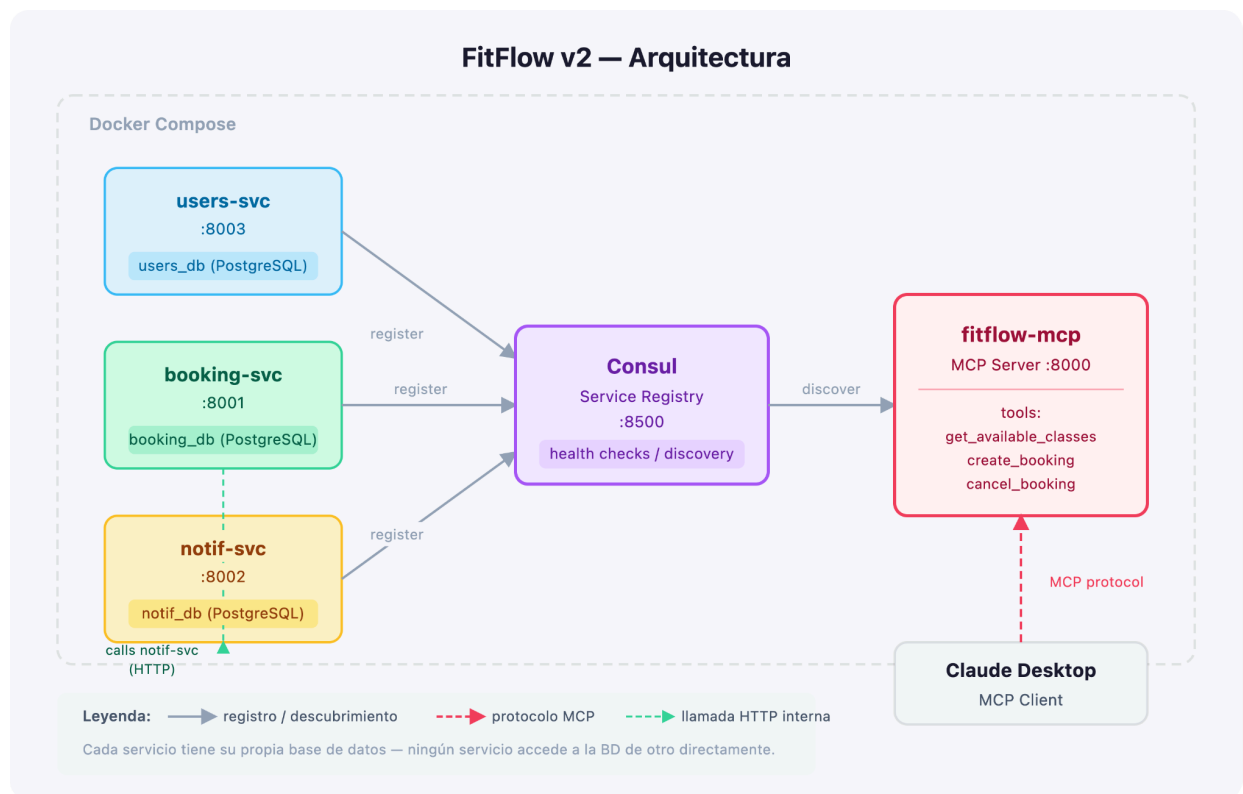


Universidad Galileo
FISICC – Postgrado en Diseño y Desarrollo de Software

Entregable final: Repositorio GitHub + README + video demo de 5–8 min

¿Qué van a construir?

FitFlow es una plataforma de reservas de clases fitness. El equipo va a construir el sistema desde cero usando una arquitectura de microservicios: tres servicios independientes que se comunican entre sí, un registro de servicios para que se descubran dinámicamente, y un servidor MCP para que un agente de IA pueda interactuar con el sistema en lenguaje natural.



Servicio	Puerto	Qué hace
users-svc	8003	Registro y autenticación de usuarios
booking-svc	8001	Gestión de reservas de clases
notif-svc	8002	Envío de notificaciones
fitflow-mcp	8000	Expone FitFlow a agentes de IA vía MCP
consul	8500	Descubrimiento y registro de servicios

Task 1 Microservicios + Docker

Qué deben lograr

Tres servicios independientes corriendo con docker compose up, cada uno con su propia base de datos.

Concepto clave: Database per Service

Cada microservicio es dueño exclusivo de sus datos. booking-svc **no puede consultar la tabla de usuarios directamente** si necesita validar un usuario, debe llamar a la API de users-svc. Esto es lo que hace a un microservicio verdaderamente independiente.

Qué debe exponer cada servicio

users-svc

- Registrar un usuario nuevo
- Login (devuelve un token JWT)
- Obtener perfil de un usuario por ID
- /healthz y /readyz

booking-svc

- Crear una reserva (requiere token JWT válido)
- Consultar una reserva por ID

- Cancelar una reserva
- Listar clases disponibles
- /healthz y /readyz

notif-svc

- Crear/enviar una notificación (puede ser solo un log por ahora)
- Consultar historial de notificaciones de un usuario
- /healthz y /readyz

Requisitos de infraestructura

- Cada servicio tiene su propio Dockerfile
- Un docker-compose.yml levanta todo el sistema
- Cada servicio se conecta a **su propia instancia de PostgreSQL**, con su propio usuario de BD (principio de menor privilegio)
- Ningún password en el código usar variables de entorno via .env (en .gitignore)
- Los servicios se comunican por **nombre lógico** (http://notif-svc:8002), nunca por IP

Pistas

- Elijan el lenguaje que mejor manejen (Python/FastAPI, Node/Express, Java/Spring Boot, Go)
- Documentación de Docker Compose: <https://docs.docker.com/compose/>
- Para el JWT en users-svc: busquen una librería JWT para su lenguaje (PyJWT, jsonwebtoken, jjwt)
- Los endpoints /healthz y /readyz son solo endpoints que devuelven {"status": "ok"} — /readyz además verifica que la BD esté conectada

Entregable

docker compose up --build

curl http://localhost:8003/healthz # → {"status": "ok"}

curl http://localhost:8001/healthz # → {"status": "ok"}

curl http://localhost:8002/healthz # → {"status": "ok"}

Task2 Service Registry con Consul + MCP Server

Qué deben lograr

Los servicios se registran solos en Consul al arrancar. Un MCP Server permite que Claude Desktop interactúe con FitFlow en lenguaje natural.

2A Auto-registro en Consul

Concepto: Consul es un Service Registry una guía telefónica de los servicios. Cuando booking-svc necesita llamar a notif-svc, no debe tener la URL hardcodeada. En cambio, le pregunta a Consul: *"¿Dónde está notif-svc en este momento?"* Consul responde con la dirección y puerto actuales.

Esto resuelve un problema real: en entornos reales (cloud, Kubernetes), las IPs cambian constantemente. El descubrimiento dinámico hace el sistema flexible.

Qué implementar:

1. Al iniciar, cada servicio se registra en Consul con: su nombre, su dirección, su puerto, y la URL de su health check
2. Consul llama a /healthz cada 10 segundos para verificar que el servicio sigue vivo
3. Si un servicio falla durante 30 segundos, Consul lo elimina del registro automáticamente
4. Cuando booking-svc necesita llamar a notif-svc, consulta a Consul para obtener la URL actual

Agregar a docker-compose.yml:

```
consul:
  image: hashicorp/consul:1.17
  ports: ["8500:8500"]
  command: agent -dev -client=0.0.0.0
```

Pistas:

- Librería cliente de Consul para Python: python-consul2. Para Node: consul. Para Java: spring-cloud-consul
- UI de Consul disponible en <http://localhost:8500> ahí pueden ver los servicios registrados con sus health checks

- El registro debe ocurrir en el arranque del servicio (al iniciar la aplicación)

2B MCP Server

Concepto: El Model Context Protocol (MCP) es un estándar abierto que permite a los agentes de IA descubrir y usar herramientas de sistemas externos. Es análogo al service registry pero para IA: así como Consul le dice a los servicios qué otros servicios existen, el MCP Server le dice al agente de IA qué acciones puede ejecutar.

Cuando conectan Claude Desktop a fitflow-mcp, Claude puede recibir instrucciones como *"reserva una clase de yoga para mañana"* y traducirlas en llamadas reales a booking-svc.

Qué implementar fitflow-mcp/server.py (o el lenguaje que elijan):

El MCP Server debe exponer al menos 3 herramientas:

Herramienta	Qué hace	A qué servicio llama
get_available_classes	Lista clases disponibles	booking-svc
create_booking	Crea una reserva	booking-svc
cancel_booking	Cancela una reserva	booking-svc

Flujo interno del MCP Server:

1. El agente de IA llama a una herramienta (ej. create_booking)
2. El MCP Server consulta a Consul para saber dónde está booking-svc
3. Hace la llamada HTTP a booking-svc
4. Devuelve el resultado al agente

Pistas:

- SDK de MCP para Python: <https://github.com/modelcontextprotocol/python-sdk>
- SDK para TypeScript: <https://github.com/modelcontextprotocol/typescript-sdk>
- Documentación general: <https://modelcontextprotocol.io/docs>
- Para conectar a Claude Desktop, configurar claude_desktop_config.json con el comando que arranca el servidor

Verificación: desde Claude Desktop, escribir *"¿qué clases hay disponibles?"* — Claude debe devolver la lista real de booking-svc.

Entregable

- `http://localhost:8500` muestra los 3 servicios en verde
- Claude Desktop puede listar clases y crear una reserva via MCP

Task 3 Resiliencia + Observabilidad

Qué deben lograr

El sistema sobrevive cuando `notif-svc` falla, y los logs permiten rastrear un request entre servicios.

3A Resiliencia en `booking-svc`

`booking-svc` llama a `notif-svc` después de crear una reserva. ¿Qué pasa si `notif-svc` está caído? Sin resiliencia: el usuario recibe un error 500 y la reserva no se crea. Con resiliencia: la reserva se crea igual, y la notificación queda pendiente para reintentar después.

Implementar al menos dos de los siguientes:

Timeout no esperar más de 2 segundos una respuesta de `notif-svc`. Si tarda más, asumir que falló.

Retries con backoff exponencial + jitter si falla, reintentar hasta 3 veces esperando progresivamente más tiempo entre intentos (0.5s, 1s, 2s) más un tiempo aleatorio pequeño. El jitter evita que todos los clientes reintenten al mismo tiempo.

Circuit Breaker si `notif-svc` falla 3 veces seguidas, el circuit breaker se "abre" y deja de intentarlo por 30 segundos (evita saturar un servicio caído). Pasados los 30 segundos, prueba una vez más. Si funciona, se "cierra". Mientras está abierto, `booking-svc` guarda la notificación como pendiente en lugar de fallar.

Pistas:

- Python: `tenacity` para retries, `pybreaker` para circuit breaker
- Node: `cockatiel` o `opossum`
- Java/Spring: `Resilience4j` (incluye todo)
- El patrón cuando el circuit está abierto se llama "outbox pattern" — guardar en BD las notificaciones pendientes para procesarlas cuando el servicio vuelva

Demo a grabar: derribar notif-svc con docker compose stop notif-svc, hacer 3+ reservas, mostrar que el sistema sigue respondiendo (no 500), mostrar el estado del circuit breaker, levantar notif-svc de nuevo.

3B Logs estructurados con x-correlation-id

Concepto: cuando un request pasa por varios servicios (usuario → booking-svc → notif-svc), necesitamos poder rastrear todo ese flujo en los logs. El x-correlation-id es un ID único que viaja en los headers HTTP de servicio en servicio.

Qué implementar:

1. Si el request llega con header x-correlation-id, usarlo. Si no, generar uno nuevo (UUID)
2. Incluir ese ID en **todos los logs** del request
3. Cuando booking-svc llama a notif-svc, enviar el mismo x-correlation-id en los headers

Resultado: en los logs pueden filtrar por un correlation_id y ver todo el viaje de ese request a través del sistema.

Los logs deben ser JSON estructurado (no texto plano), con al menos: correlation_id, service, event, level, timestamp.

Pistas:

- Python: structlog
- Node: pino
- Java: logback con formato JSON + MDC para el correlation ID

Entregable

- Video mostrando el circuit breaker en acción
- Logs JSON con correlation_id rastreable entre servicios

Task 4 Seguridad + README + Demo

Qué deben lograr

JWT funcionando en todos los endpoints protegidos, secretos fuera del código, README completo y video grabado.

4A JWT en todos los servicios

users-svc ya emite el token en el task 1 1. Ahora booking-svc debe validarlo en cada request protegido.

- El token debe incluir al menos el user_id del usuario
- Los endpoints de booking-svc que modifican datos deben requerir token válido
- Si el token está expirado o es inválido, responder 401 Unauthorized
- El user_id del token debe aparecer en los logs junto al correlation_id

4B Gestión de secretos

Verificar que ningún secreto esté en el código ni en el repositorio:

- Passwords de BD en .env (en .gitignore)
- JWT_SECRET en .env
- Documentar en el README cómo rotar credenciales (pasos concretos para cambiar un password sin downtime)

4C README

El README debe tener estas secciones:

Arquitectura — diagrama del sistema (puede ser ASCII art como el de este documento)

Cómo correr el proyecto

```
git clone <repo>
```

```
cp .env.example .env # completar los valores
```

```
docker compose up --build
```

Video Checkpoint

1. docker compose up → mostrar Consul en localhost:8500 con los 3 servicios en verde
2. Registrar un usuario → hacer login → mostrar el JWT recibido
3. Crear una reserva usando el JWT → ver el log JSON con correlation_id
4. Derribar notif-svc → hacer reservas → mostrar que el sistema sigue respondiendo → mostrar circuit breaker abierto
5. Levantar notif-svc → circuit breaker se cierra

6. Abrir Claude Desktop → "*¿Qué clases hay disponibles?*" → Claude llama `get_available_classes` vía MCP
7. "*Reserva yoga para el viernes*" → Claude llama `create_booking` vía MCP → mostrar la reserva creada en la BD

Task 5 Agent-to-Agent (A2A)

Qué deben lograr

Reemplazar la interacción directa usuario → MCP Server por una red de agentes especializados que se delegan tareas entre sí usando el protocolo A2A.

Concepto clave: MCP vs A2A

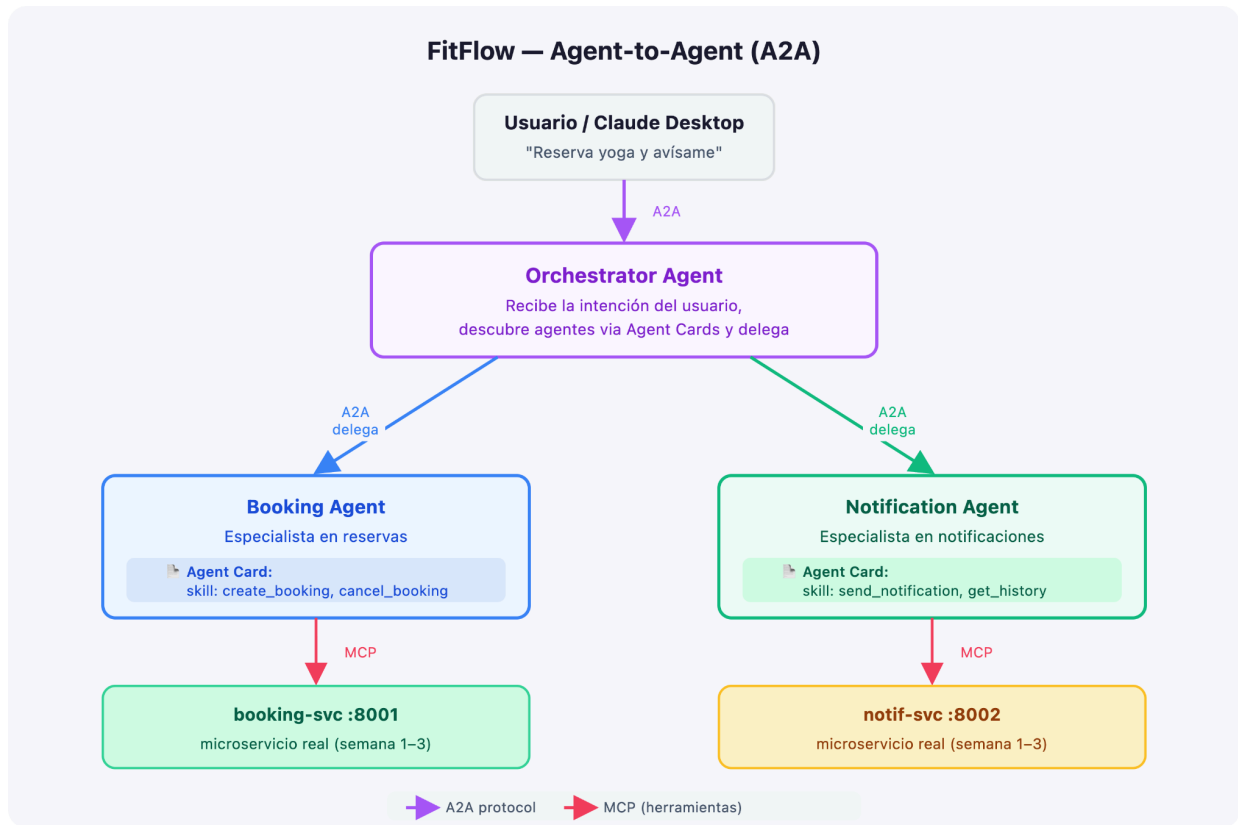
Hasta ahora usaron **MCP** para que Claude (un agente) hable con FitFlow (herramientas/servicios). MCP responde la pregunta: *¿cómo un agente usa un sistema externo?*

A2A (Agent-to-Agent, protocolo abierto de Google) responde otra pregunta: *¿cómo un agente delega trabajo a otro agente?* En lugar de un solo agente que lo sabe todo, se tienen agentes especializados que se descubren y se coordinan entre sí.

Analogía directa con microservicios:

Microservicios	Agentes con A2A
Cada servicio tiene una responsabilidad	Cada agente tiene una especialidad
Se registran en Consul (service registry)	Se publican con un Agent Card (agent registry)
Se descubren dinámicamente	Se descubren via Agent Cards
Se comunican por HTTP	Se comunican por A2A protocol

Arquitectura con A2A



Qué implementar

Agent Cards: cada agente publica un archivo JSON que describe sus capacidades, de forma análoga a como un microservicio publica sus endpoints. Este JSON se llama Agent Card y es el mecanismo de descubrimiento de A2A.

Ejemplo de Agent Card para el Booking Agent:

```
{  
  
  "name": "FitFlow Booking Agent",  
  
  "description": "Gestiona reservas de clases fitness en FitFlow",  
  
  "url": "http://booking-agent:9001",
```

```

"skills": [

  {

    "id": "create_booking",

    "name": "Crear reserva",

    "description": "Reserva una clase para un usuario en una fecha específica"

  },

  {

    "id": "cancel_booking",

    "name": "Cancelar reserva"

  }

]

}

```

Orchestrator Agent: recibe la instrucción del usuario, decide qué agente(s) necesita, los descubre por su Agent Card, y les delega la tarea. Si la tarea requiere reservar Y notificar, coordina ambos agentes en secuencia.

Booking Agent: agente especializado en operaciones de reserva. Internamente usa el MCP Server de FitFlow para ejecutar las acciones reales.

Notification Agent: agente especializado en notificaciones. También usa MCP internamente.

Recursos

- Especificación A2A: <https://google.github.io/A2A/>
- Python SDK: <https://github.com/google/a2a-python>
- Ejemplos: <https://github.com/google/a2a-samples>

Demo

1. Instrucción en lenguaje natural al Orchestrator: *"Reserva yoga para el viernes y avísame por notificación"*
2. Orchestrator descubre Booking Agent y Notification Agent por sus Agent Cards

3. Delega create_booking al Booking Agent → Booking Agent llama FitFlow via MCP
4. Delega send_notification al Notification Agent → Notification Agent llama FitFlow via MCP
5. Mostrar los logs de comunicación A2A entre agentes

Qué agregar al docker-compose

Tres contenedores nuevos: orchestrator-agent, booking-agent, notification-agent, cada uno con su propio puerto y publicando su Agent Card en /.well-known/agent.json.

Checkpoint task 5

- Los tres agentes corren como contenedores
 - Demo del flujo completo: usuario → Orchestrator → Booking Agent + Notification Agent → microservicios
 - README con sección "Agent-to-Agent" explicando la diferencia entre MCP y A2A
-

Criterios de evaluación

Criterio	Puntos
Los 3 servicios corren con docker compose up y tienen DB propia	20
Los servicios se registran en Consul y se descubren dinámicamente	20
MCP Server funciona: Claude puede crear una reserva	20
Circuit breaker demostrado: notif-svc cae, sistema sigue	20
JWT implementado + secretos fuera del código	10
Ageng 2 Agent Implementation	20
Total	110

Puntos extra Despliegue en Cloud (+15 pts)

En lugar de (o además de) correr todo en Docker Compose local, desplegar el sistema en un proveedor de nube.

Opciones sugeridas:

AWS (Free Tier):

- Los servicios como contenedores en **ECS Fargate** o en una instancia **EC2** con Docker Compose
- Bases de datos en **RDS PostgreSQL**
- Secrets en **AWS Secrets Manager** en lugar de .env
- Para Consul: puede correr como contenedor en el mismo cluster, o reemplazarlo por **AWS Cloud Map** (el service registry nativo de AWS)

Google Cloud (Free Tier):

- Contenedores en **Cloud Run** (serverless, escala a cero — ideal para demos)
- Bases de datos en **Cloud SQL**
- Secrets en **Secret Manager**

Railway / Render / Fly.io (más simple, recomendado si es primera vez):

- Plataformas que aceptan un docker-compose.yml directamente o despliegan desde GitHub
- Railway y Render tienen free tier suficiente para este proyecto
- Fly.io tiene buen soporte para múltiples servicios con networking interno

Qué documentar si eligen cloud:

- URL pública de cada servicio (o del API Gateway si usan uno)
- Cómo se manejan los secretos en ese entorno (no .env, sino el servicio de secrets del proveedor)
- Screenshot o link a la UI de Consul (o del service registry equivalente) en la nube
- El docker-compose.yml local sigue siendo válido para desarrollo — el README debe explicar cómo correr en ambos entornos

Criterio de evaluación puntos extra:

Criterio	Puntos extra
Sistema accesible por URL pública (todos los servicios)	+8
Secretos manejados con el servicio de secrets del proveedor (no .env)	+4
README explica cómo desplegar en cloud paso a paso	+3
Total extra	+15